YOLO26n for Infrared Small-UAV Detection: Baseline Experiments

Yaorong Huang, Chunshen Yu, Yifan Zhai, and Zhiwen Luo
Macao Polytechnic University

***Abstract*—This report presents a YOLO26n baseline for single-class infrared UAV bounding-box detection on SIRST-UAVB. An audit of the released images found exact-pixel overlap across the original training and validation split and conflicting annotations for some identical images. We fixed a background-view holdout containing 1,438 training and 505 validation images after duplicate removal and annotation quarantine. Training completed 92 full epochs and part of epoch 93 within a fixed execution window. Final validation of the selected checkpoint achieved 66.5% precision, 49.2% recall, 56.2% mAP@0.5, and 23.2% mAP@0.5:0.95 on 491 UAV instances. These measurements establish the baseline for subsequent shallow pinwheel-shaped convolution and high-resolution P2 experiments.**

***Index Terms*—Infrared UAV detection, YOLO26n, small targets, dataset audit, baseline experiments.**

I. Background and Study Design

Small infrared UAVs contain few pixels and limited texture. Bright background structures and birds can resemble the target, while small coordinate errors strongly affect box overlap. We study bounding-box detection using YOLO26n as the lightweight baseline [1], [2]. Pinwheel-shaped convolution (PConv) provides directional shallow sampling for infrared small-target features [3]; a stride-4 P2 prediction branch provides a second way to preserve spatial detail.

The implementation contains B0 (baseline), B1 (a layer-1 PConv replacement), and B2 (an added P2 branch). The experiment reported here trains B0. B1 and B2 currently have model-construction and checkpoint-restoration checks; their detection improvements remain to be measured using the same fixed data protocol.

II. Dataset Audit and Preparation

SIRST-UAVB is the main dataset [3]. The acquired archive contains 3,000 grayscale images at 640 by 512 pixels. Direct inspection of its native labels counted 2,961 class-0 and 1,747 class-1 boxes. We identified class 0 as UAV and class 1 as bird by cross-checking boxes against masks and the authors' description that bird targets are excluded from masks. Positive mask pixels overlapped 2,955 of the class-0 boxes and only one class-1 box, with two pixels in that exceptional overlap. Native boxes were retained rather than regenerated from masks.

The original 2,400/600 split has 496 exact-pixel groups shared across training and validation. Three additional pixel-identical groups have inconsistent labels. We isolated those conflicts, the six UAV images without corresponding positive mask pixels and their pixel-identical copies, and close background views at the split boundary. Exact-pixel deduplication and the fixed background-view review produced the counts in Table I. Bird boxes are retained in the audit manifest but omitted from the single-class UAV training labels.

TABLE I
Fixed SIRST-UAVB Background-View Holdout

Quantity	Training	Validation
Images	1,438	505
Native UAV boxes	1,421	491
Bird boxes retained for analysis	925	227

The release provides no original video or session identifiers. The grouping therefore represents inferred background-view families, with nearby facade views held out for validation. It has zero cross-split exact-pixel duplicates and no independent test split. The image list, class map, grouping policy, and prepared-manifest digest are frozen in the repository. Conversion preserves original pixels and dimensions; training then uses aspect-ratio-preserving resizing and padding to a 640-pixel input.

III. Model and Experimental Settings

B0 uses the n-scale, single-class YOLO26 architecture with prediction strides 8, 16, and 32 and its end-to-end output branch. The constructed training model has 2,504,190 parameters. For consistent initialization across future variants, we transfer only same-name, same-shape tensors in layers 0–16 from the official YOLO26n checkpoint: 348 tensors were copied. Layers from 17 onward are randomly initialized. Thus the experiment uses a shared-prefix transfer policy rather than full-model pretrained initialization.

Training ran on one RTX 2080 Ti with Python 3.10.8, PyTorch 2.1.2+cu118, and Ultralytics 8.4.184. Table II records the effective settings. The automatic optimizer resolved to AdamW at an initial learning rate of 0.002; this effective value supersedes the generic default in the argument file. Standard training augmentation includes mosaic, scaling, translation, and horizontal flipping. Validation uses the fixed 505-image split without training augmentation.

A dual-clock stopping callback reserved time for saving and final validation. The framework printed "93 epochs completed in 0.946 hours," but the last visible training progress in epoch 93 was 317/360 batches before the time stop. We therefore count 92 complete epochs plus a partial 93rd epoch. No model training was performed on the local computer. The selected checkpoint was saved at epoch 81, as confirmed by its stored epoch index and the validation CSV.

0000–0000/00$00.00 © 2021 IEEE

TABLE II
EFFECTIVE BASELINE TRAINING SETTINGS

Setting	Value
Input / batch size	640 pixels / 4
Optimizer / initial learning rate	AdamW / 0.002
Weight decay	0.0005
Seed / workers	42 / 4
Mixed precision	Enabled; AMP check passed
Epoch cap / full epochs completed	300 / 92
Final partial epoch	93
Checkpoint selection	Highest validation mAP@0.5:0.95

TABLE III
FINAL VALIDATION ON 505 IMAGES AND 491 UAV BOXES (%)

Model state	Precision	Recall	AP50	AP50:95
Partial epoch 93	72.4	48.3	52.6	20.6
Selected best (epoch 81)	66.5	49.2	56.2	23.2

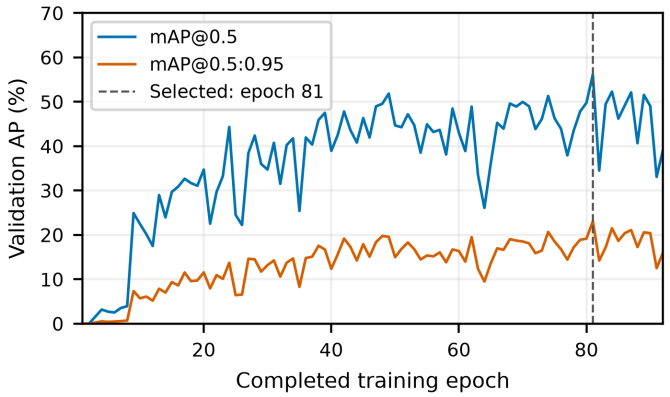


Fig. 1. In-training validation AP from the verified CSV snapshot, through complete epoch 92. The dashed line marks the selected checkpoint at epoch 81. The partial final epoch and post-training best-checkpoint revalidation are reported separately in Table III.

IV. RESULTS AND ANALYSIS

Table III presents the final end-of-run validation output. Precision and recall are the operating point reported by the Ultralytics validator, and mAP@0.5:0.95 averages AP across IoU thresholds from 0.50 to 0.95. The values are rounded to the three-decimal precision of the final console output. The selected checkpoint was revalidated after optimizer stripping; its final scores differ slightly from the in-training epoch-81 values of 56.176% mAP@0.5 and 23.061% mAP@0.5:0.95.

The selected checkpoint has higher AP than the final partial-epoch state, supporting validation-based checkpoint selection. Recall of 49.2% leaves substantial missed-target headroom. The gap between AP50 and AP50:95 also motivates closer analysis of localization at small target scales. Figure 1 shows sizeable validation variation across epochs; this single-seed run establishes a reference rather than a measured improvement for PConv or P2. A later comparison should keep the data, initialization policy, and training schedule matched.

A. Timing and Artifact Verification

The final GPU validation reported 0.1 ms preprocessing, 3.1 ms inference, and 0.1 ms postprocessing per image, using validation batches of eight. These are framework timings for offline images. Camera input, video decoding, display, and device transfer require a separate end-to-end benchmark on the deployment device.

The cloud log confirms that both best.pt and last.pt were written and optimizer-stripped to approximately 5.4 MB before final validation returned. The independently hashed local archive was captured just before shutdown: it contains the selected epoch-81 best checkpoint (25,780,216 bytes, including training state) and an epoch-92 last checkpoint (25,781,880 bytes). Both deserialize successfully and each has 708 state tensors with finite floating-point values. Both final stripped files have now been retrieved: each is 5,388,158 bytes, and each local SHA256 matches both the pre-shutdown artifact manifest and a fresh cloud SHA256. Both deserialize with finite floating-point state tensors. Every final-best tensor equals the archived epoch-81 EMA tensor; final-last differs from the epoch-92 snapshot. Optimizer stripping resets the stored epoch to -1, so training progress is identified from the final 93-row CSV, stopping record, and console log. The last CSV row follows the partial epoch; Fig. 1 retains only the 92 complete epochs. The retrieval session used no GPU and was stopped after transfer.

V. EDGE DEPLOYMENT AND FURTHER EXPERIMENTS

The next algorithm experiments replace the second backbone convolution with PConv and add a separate stride-4 P2 branch. Their relative recall, AP, and inference cost will be compared with B0 under a matched protocol. Bird-box matching and target-size groups will provide more specific error analysis.

YOLO26n conversion for H10, Raspberry Pi execution, and real infrared-video timing have not yet been measured for this trained baseline. The deployment stage will record the compiler/runtime combination, conversion success, any accuracy change, and end-to-end video latency. The GPU validation timing above is the current measured reference.

VI. CONCLUSION

We completed a real-data YOLO26n baseline with a fixed, audited background-view holdout. The selected epoch-81 checkpoint achieved 56.2% mAP@0.5 and 23.2% mAP@0.5:0.95 after 92 full training epochs and part of epoch 93. Local checkpoint integrity and the final cloud save messages were checked, and the cloud platform was confirmed stopped after the run. The remaining study compares shallow directional sampling and higher-resolution prediction using this baseline and then measures deployment performance.

REFERENCES

[1] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," arXiv:2606.03748, 2026. [Online]. Available: https://arxiv.org/abs/2606.03748

[2] Ultralytics, "YOLO26 model documentation," 2026, accessed: Oct. 23, 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo26/

[3] J. Yang, S. Liu, J. Wu, X. Su, N. Hai, and X. Huang, "Pinwheel-shaped convolution and scale-based dynamic loss for infrared small target detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 9, 2025, pp. 9202–9210. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/32996